

```
        // If the puzzle is solved, return True:
        undoMove(board, move); // Reset to the original puzzle.
        return true;
    }
}
```

---

The `for` loop sets the move variable to each of the directions returned by `getValidMoves()`. For each move, we call `makeMove()` to modify the board data structure with the move and to add the move to the list or array in `movesMade`.

Next, the code recursively calls `attemptMove()` to explore the range of all possible future moves within the depth set by `movesRemaining`. The board and `movesMade` variables are forwarded to this recursive call. The code sets the recursive call's `movesRemaining` parameter to `movesRemaining - 1` so that it decreases by one. It also sets the `prevMode` parameter to `move` so that it doesn't immediately undo the move just made.

If the recursive call returns `True`, a solution exists and is recorded in the `movesMade` list or array. We call the `undoMove()` function so that board will contain the original puzzle after the execution returns to `solve()` and then return `True` to indicate a solution has been found.

The Python code for `attemptMove()` continues as follows:

### **Python**

---

```
# Undo the move to set up for the next move:
undoMove(board, move)
movesMade.pop() # Remove the last move since it was undone.
return False # BASE CASE - Unable to find a solution.
```

---

The JavaScript code is as follows:

### **JavaScript**

---

```
// Undo the move to set up for the next move:
undoMove(board, move);
movesMade.pop(); // Remove the last move since it was undone.
}
return false; // BASE CASE - Unable to find a solution.
}
```

---